

SGOV HANDOVER

GCON 2027 Nursing Intake Platform

Step-by-step instructions for opening, running and checking the application in Microsoft Visual Studio 2022.

Project folder: C:\GCRA\nurse
Prepared: 8 September 2026

Purpose

Use this guide to open the existing source folder in Visual Studio, start the local Node API and React/Vite frontend, verify the profile workflow and understand the difference between the local application and the public demonstration site.

What this project contains	How it is started
React and Vite frontend Node.js local API adapter Leaflet and OpenStreetMap address picker Applicant, Admissions and Employer demo surfaces	Terminal 1: npm run dev:api Terminal 2: npm run dev -- --host 127.0.0.1 --port 4173 Browser: http://127.0.0.1:4173/

1. Before opening the project

The application is a folder-based React/Vite and Node.js project. It does not require a Visual Studio solution file. Visual Studio is used as the editor, integrated terminal and debugging workspace.

Install or confirm the following on the developer PC:

- Visual Studio 2022 Community, Professional or Enterprise.
- Node.js 20 LTS or newer, with npm available in a terminal.
- The Visual Studio Node.js development tools workload if offered by the installer. The ASP.NET and web development workload is optional for this project.
- A modern browser such as Microsoft Edge or Google Chrome.
- Access to the project folder C:\GCRA\nurse or a clone of the repository.
- Network access to the OpenStreetMap tile and Nominatim endpoints when testing the address map.

Important

Open the project root C:\GCRA\nurse. Do not open only the dist folder. The source files, package-lock.json, server.mjs, data adapter and Vite configuration are all needed for local development.

Confirm Node.js and npm

```
node --version  
npm --version
```

Restore the exact locked dependencies from the project root:

```
npm ci
```

Use npm install only if package-lock.json is intentionally being regenerated. For a handover checkout, npm ci is preferred because it uses the committed lock file.

2. Open the application in Visual Studio

Follow these steps on the handover PC:

1

Start Visual Studio 2022

Open Visual Studio from the Windows Start menu. You do not need to create a new project.

2

Open the folder

Choose File > Open > Folder, browse to C:\GCRA\nurse, and select the folder. If Visual Studio asks whether to trust the folder, confirm it only when the source came from the approved handover location.

3

Use Solution Explorer

The folder should appear in Solution Explorer. There is no .sln file in this project; that is expected for a Vite application.

4

Open the integrated terminal

Choose View > Terminal. Confirm the prompt is at C:\GCRA\nurse. If it opens elsewhere, change to the project root before running commands.

5

Create two terminal sessions

Use the plus button in the Terminal panel to open a second terminal. The API and frontend must run at the same time.

Visual Studio does not start this app with F5 by default

This checkout has no Visual Studio solution or launch profile. Start the two processes from the integrated terminal. Once running, use the browser URL printed by Vite for the rendered app.

Optional: open source files directly from Solution Explorer

- src/main.jsx - main React workflow and page composition.
- src/AddressMapPicker.jsx - Leaflet map, OpenStreetMap tiles and reverse geocoding.
- src/qualification.js - qualification pathways and minimum checks.
- server.mjs and server/ - local API, authentication, storage and workflow rules.
- vite.config.js - frontend dev server and API proxy settings.

3. Start the application

Run the following commands in the two Visual Studio terminal sessions.

Terminal 1 - local API

```
npm run dev:api
```

A successful start shows the local API listening on `http://127.0.0.1:4000`.

Terminal 2 - React/Vite frontend

```
npm run dev -- --host 127.0.0.1 --port 4173
```

Open the exact URL Vite prints, normally:

```
http://127.0.0.1:4173/
```

If port 4173 is already in use, start Vite on 4174 instead and use the URL it prints:

```
npm run dev -- --host 127.0.0.1 --port 4174
```

First smoke check

1

Open the home page

The title should be GCON 2027 - Student Nurse Intake and the landing page should render without a blank screen.

2

Check the API

Open `http://127.0.0.1:4000/api/health` in another browser tab. The response should show `ok: true` and `service: gcon-local-api`.

3

Use a demo role

Choose Login, then use the clearly labelled Applicant demo, Admissions demo or Employer demo shortcuts for presentation and QA.

Stop the app

Return to each terminal and press `Ctrl+C`. Stop both the frontend and API before closing Visual Studio or switching branches.

4. Verify the profile and map

Use this short acceptance path after starting the app:

- Applicant portal -> Check my qualification -> Check qualification.
- Continue to Access management and enter a temporary local learner account if required.
- Open the profile and scroll to Gauteng address.
- Confirm the map shows street tiles, the pin is visible, the zoom controls work and the OpenStreetMap attribution is present.
- Click or drag the pin and confirm the address status changes while the selected province is checked.
- In School details, add a remaining subject and use its Remove button to confirm an incorrect entry can be deleted.

Map network note

The map first uses `https://{s}.tile.openstreetmap.org`. If the browser cannot complete that TLS connection, the application automatically switches to `https://{s}.tile.openstreetmap.de`. Both are OpenStreetMap tile endpoints and the map retains the OpenStreetMap contributor attribution.

If the map is gray

- Open the browser developer tools and check for failed tile requests or `ERR_SSL_VERSION_OR_CIPHER_MISMATCH`.
- Ask the network administrator to allow the OpenStreetMap tile hosts and `nominatim.openstreetmap.org`, or provide approved `VITE_OSM_TILE_URL` and `VITE_OSM_FALLBACK_TILE_URL` values.
- Confirm that the map container is visible and that the marker and Leaflet controls render. A marker with no tiles usually indicates network or TLS filtering, not a React layout failure.

Automated verification

```
npm test
npm run build
```

The test suite covers API, authentication, workflow, qualification and export behavior. The production build compiles the Vite frontend and generates the Sites worker output.

5. Visual Studio and deployment caveats

Keep these points with the project when handing it to another developer.

Local development state

- The local API uses the encrypted development store under `data/gcon-local-store.enc` and local document bytes under `data/documents/`.
- Do not commit uploaded documents, local secrets or real applicant data.
- Copy `.env.example` to an environment-specific file only when needed. Never commit production connection strings or keys.

Production integrations

- `DATABASE_URL` is the production PostgreSQL connection setting.
- `AZURE_STORAGE_CONNECTION_STRING` is for private document storage.
- `AZURE_KEY_VAULT_URI` and an external identity provider are required for production secrets and authentication.
- The public ChatGPT Sites URL is a demonstration surface. It does not use the local encrypted API or production database.

Useful endpoints

Endpoint	Use
<code>http://127.0.0.1:4173/</code>	Local frontend
<code>http://127.0.0.1:4000/api/health</code>	API health check
<code>http://127.0.0.1:4000/v1/openapi.json</code>	Machine-readable API contract
<code>https://gcon-nursing-intake.alistairjohanson.chatgpt.site</code>	Public demonstration site

Handover checklist

The developer should be able to open `C:\GCRA\nurse`, run `npm ci`, start both terminals, open the local URL, check the API health response, exercise the applicant profile and confirm the map tiles load. If any of those steps fails, record the exact terminal or browser error before changing configuration.